

Coug Assets

KADEN METZGER & TRAVIS TAKUSHI

SEPTEMBER 29TH, 2026

A solid orange horizontal bar spanning the width of the slide at the bottom.

Coug Asset Objectives

- Create a working web application that EECS staff and admin can use
- Streamline the asset sharing process within the EECS department
- Create an efficient and easy to use system

Problem Statement & Scope

- The EECS department at WSU has assets that are shared within their department. Often times, sharing these assets can be difficult. The current system in place is outdated and inefficient, causing some assets to be purchased that are already present. Our project aims to solve this problem.
- Our target audience is anyone within the WSU EECS department whether they are teachers or students.

Technologies Used

- Coug Assets is being developed with Next.js and PostgreSQL for the our database
- Next.js is a react based framework
- Next.js and react use something known as hooks and states for UI rendering
- Next.js aims to collapse the full stack dev experience into a singular layer for simpler, more efficient development

Functional Requirements

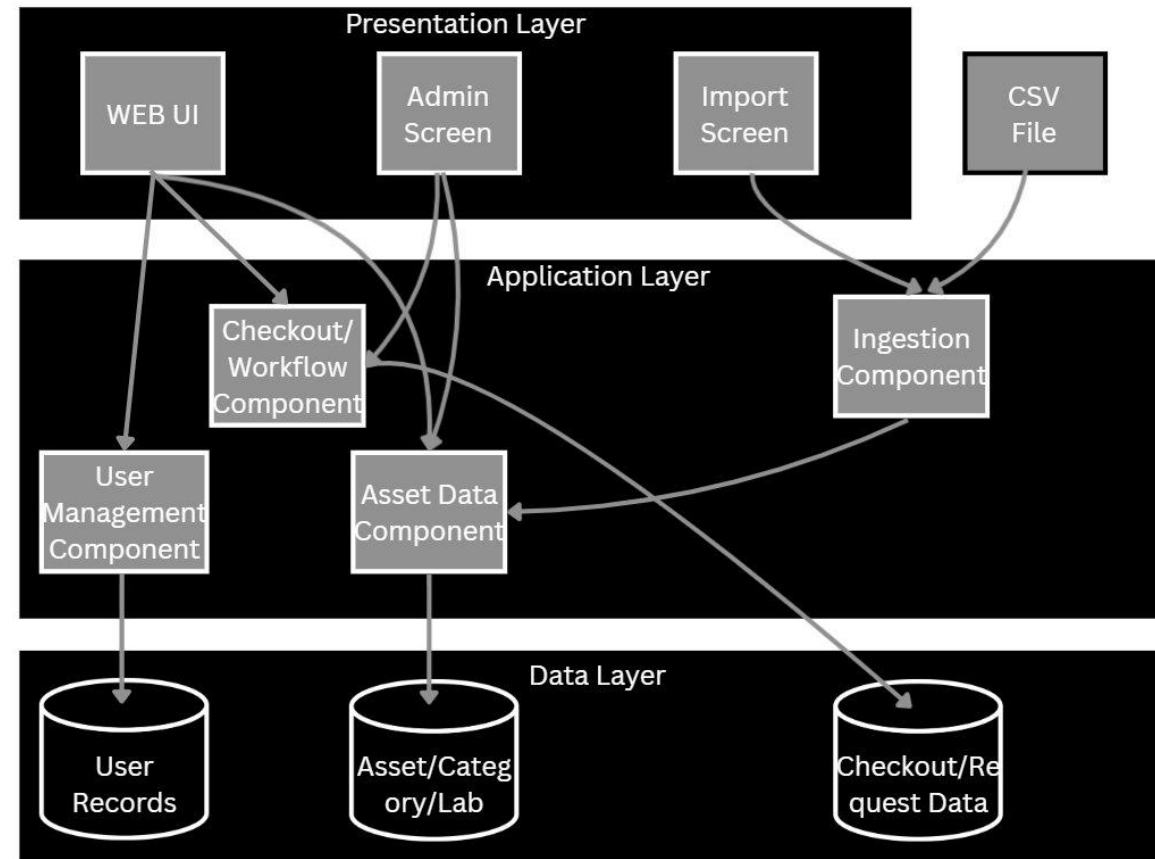
- User Login
- Asset Search
- Asset Checkout
- Admin Dashboard
- Interfacing with Database
- Routing
- Security

Non-Functional Requirements

- Usability
- Security
- Performance
- Documentation

System Architecture

- UML Diagram



System Architecture

- Our architecture features 3 layers, UI or presentation layer, application layer, and the database layer
- Our main flow:
 - User interacts with the UI layer
 - UI layer directly runs the application layer software
 - Application layer communicates with the database layer
 - Database layer fetches, updates, removes, etc entries from the database depending on the function called

Sprint 4 Achievements

- Created owner user type with elevated privileges to control user creation and access
- Created unit tests to guarantee health of endpoints
- Created system test to ensure full system workflow works
- Refactored Database Schema for clarity and ease of use
- Refactored UI elements to make them cleaner as well as scale better
- Integrated both group members work together

System demonstration

- Admin dashboard
 - Approve, Deny, Mark return
 - Create/Edit assets
 - Search existing assets
 - Added category tags so multiple categories can be added to one asset
- Homepage
 - Search/Filter Assets
 - Create asset request
 - View asset details
 - View current checked out assets
 - View Current Checked out requests
 - For admin
 - Admin page
 - Viewing users
 - Owner Page
 - Creating and deleting users
 - Upload page
- Login
 - Basic login functionality for all different admin types

Testing

- Unit tests and system test both use vitetest
- Unit tests call route handlers directly
 - `docker compose --profile test run --rm test`
- System test calls http fetch directly from app
 - `docker compose --profile system-test up --build --abort-on-container-exit --exit-code-from system-test system-test`

Responsible Use of AI

- Use of chatgpt and github copilot to ask questions when stuck, as well as proper formatting and optimization of systems

Plan and Deliverables for Next Sprint

- Functional Requirement Plans:

- Finish Homepage and Admin Dash
- Handle soon to be due and overdue assets
- Fix any issues regarding database refactoring (CSV import)
- Implement barcode scanning
- Polish all features and conduct tests to help identify bugs
- Perform Security Testing

- Non-Functional Requirement Plans:

- Redesign the Admin Approve/Deny menu to prep for scaling issues
- Polish UI
- Discuss with client about the web application's accessibility and ease of use

Next Semester Timeline and Deliverables

- Timeline:

- Sprint 4: Finish all main functionality and conduct all testing phases
- Sprint 5: Integrate with OKTA and VM, polish UI, keep refining the site
- Sprint 6: Add final touches, recap functional and non-functional requirements to ensure clients needs are met, and finally deliver a fully working product

- Final Deliverables:

- A working product that is accessible and intuitive to use
- Our site must be integrated with OKTA and usable by staff and students
- Fully fleshed out features that make the site easy to use and manage

Conclusion

- Our product is up to par with client standards so far
- Created a minimum viable product that can be tested with users
- We have made good progress and remain on track to successfully finish the app